

# mfsk-core

[github.com/jl1nie/mfsk-core](https://github.com/jl1nie/mfsk-core) · [crates.io/crates/mfsk-core](https://crates.io/crates/mfsk-core) · GPL-3.0-or-later

WSJT-X のデコーダを、まるごと Rust で書き直しました。  
いま目の前の小さな箱が、それで復調しています。

FT8 · FT4 · FST4 · WSPR · JT9 · JT65 · Q65 · MSK144

8つのモードファミリを、ひとつのライブラリで。デスクトップにも、ブラウザにも、スマホにも、マイコンにも。

## WSJT-X と同等

実録音での再現率も、AWGN 感度も、  
実バイナリと突き合わせて確認しています

## 1つのソース

デスクトップ / WASM / Android・iOS /  
no\_std マイコン — 同じコードが動きます

## 約150 KB で動く

FT8 の復調に必要な RAM。  
実機で、実際に電波を復調しています

JL1NIE / 友部 稔

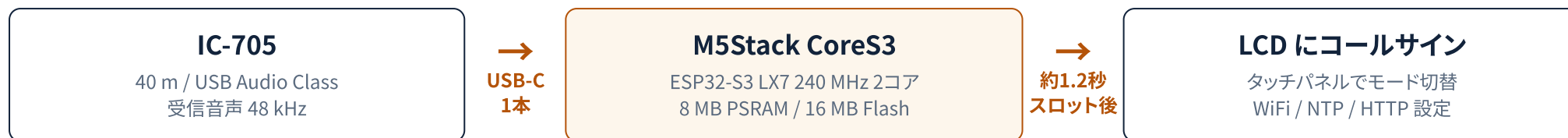
デモ機: M5Stack CoreS3 + IC-705 (USB Audio 直結、PC なし)



[github.com/jl1nie/mfsk-core](https://github.com/jl1nie/mfsk-core)

## 02 デモ:いま目の前で起きていること

M5Stack CoreS3



この経路に PC は 1 台も入っていません。復調はすべて CoreS3 の中で完結しています。

実測 (2026-08-23、IC-705 実機 40 m)

**6-8 局**

1 スロット (15 秒) あたりの  
FT8 デコード数

**-24 dB**

到達した最弱信号  
(+8 ~ -24 dB)

同日、WSPR も slot 1 src=uac decoded 1 station(s) を記録。  
USB Audio 取り込み自体は 10 分間・125 MB 連続無エラー、WiFi  
接続を維持したまま。

1つのイメージに、4つの受信機

**uac**

無線機の USB  
Audio から FT8

**wspr**

WSPR 受信・スポ  
ット一覧

**fst4**

FST4 ワイドバンド  
監視

**decode**

内蔵 WAV のデモ  
(無線機不要)

どれを動かすかは NVS に保存され、**タッチパネルで切り替わります**。書き換え不要 — USB ホスト  
を起動するとフラッシュが使うポートを奪ってしまうので、これは仕様上どうしても必要でした。

**電源のきまり:** USB-C は「もらう」と「配る」を同時にできません。起動時に VBUS があれば ペリフェ  
ラル (充電・書き込み・シリアル)、なければ USB ホスト (無線機を鳴らす)。役割は起動の瞬間に決  
まります。

WSJT-X と同じアルゴリズムが、手のひらの ESP32 の中で、無線機から直接受けた音を復調しています。

### WSJT-X が素晴らしいこと

- K1JT Joe Taylor 氏らによる、実戦で鍛えられた実装
- 本プロジェクトのプロトコル定数は、すべてここが出典
- デスクトップでは徹底的に最適化されている
- **標準実装の座は、これからも WSJT-X のもの**

### それでも困ること

- Fortran + C + Qt の混成デスクトップアプリ
- ブラウザ (WASM) には載らない
- Android / iOS で Fortran ランタイムを積むのは非現実的
- マイコン (no\_std) はそもそも土俵の外



**アルゴリズムはそのまま。載る場所を増やす。**

### 1:1 の移植

全アルゴリズムファイルが、移植元の `lib/*.f90` / `lib/*.c` を明記しています。意図的に WSJT-X と変えた箇所は、必ずコメントでそう書く決まりです。

### 再現性で守る

WSJT-X 配布の実録音に対する golden テストと、AWGN 掃引による感度測定。再現率 (recall) だけでなく誤デコード (precision) も常に測ります。

### 依存を持たない

純 Rust。FFT すら外部から差し替え可能。  
`no_std + alloc` でビルドでき、C から呼べる ABI を用意しています。

**標準実装を置き換えるのではなく、標準実装が行けない場所へ持っていくためのライブラリです。**

| モード                            | スロット      | 誤り訂正                                     | メッセージ  | 同期                | feature |
|--------------------------------|-----------|------------------------------------------|--------|-------------------|---------|
| FT8                            | 15 s      | LDPC(174, 91) + CRC-14                   | 77 bit | Costas-7 × 3      | ft8     |
| FT4                            | 7.5 s     | LDPC(174, 91) + CRC-14                   | 77 bit | Costas-4 × 4      | ft4     |
| FST4<br>-15/-30/-60A/-120/-300 | 15-300 s  | LDPC(240, 101) + CRC-24                  | 77 bit | Costas-8 × 5      | fst4    |
| WSPR                           | 120 s     | 畳み込み $r=\frac{1}{2}$ K=32 + Fano         | 50 bit | シンボル毎 LSB (npr3)  | wspr    |
| JT9                            | 60 s      | 畳み込み $r=\frac{1}{2}$ K=32 + Fano         | 72 bit | 分散 16 スロット        | jt9     |
| JT65                           | 60 s      | Reed-Solomon(63, 12) GF(2 <sup>6</sup> ) | 72 bit | 分散 63 スロット        | jt65    |
| Q65 -30A / -60A・・E             | 30 / 60 s | QRA(15, 65) GF(2 <sup>6</sup> ) + CRC-12 | 77 bit | 分散 22 スロット        | q65     |
| MSK144                         | 15 s      | LDPC(128, 90) + CRC-13                   | 77 bit | 流星バースト走査 (整合フィルタ) | msk144  |

### デコードもエンコードも

各モードとも、復調 (デコード) だけでなく変調波の生成 (シンセサイズ) も実装しています。テストは「自分で作った波形を自分で復調する」だけでは信用しない方針で、**WSJT-X が配布している実際の録音**に対する既知の正解デコードと突き合わせます。

### MSK144 だけは別扱い

MSK144 は連続位相の2値 MSK で、他モードのような「固定スロット」の枠に収まりません。そのため共通の Protocol トレイトには載せず、専用の復調器を持ちます。これは埋め残しではなく、設計上の判断です。

FST4 は5つの T/R 周期サブモード、Q65 は 30 秒 1 + 60 秒 EME 用 5 のサブモードを持ちます。同じ FEC・メッセージ符号・同期配置を共有し、NSPS とトーン間隔だけが異なります。

8つのモードファミリ。デコードもエンコードも、すべて同じ1つのライブラリの中にあります。

| モード       | 実録音での再現率                    | 誤デコード           | AWGN 感度の WSJT-X 比                                       |
|-----------|-----------------------------|-----------------|---------------------------------------------------------|
| FT8 (ホスト) | 8/8 (WSJT-X) , 18/18 (JTDX) | 裏付けのない検出 0      | 約 -21.6 dB (WSJT-X は -20~-21 dB)                        |
| FT4       | 6/6                         | 0               | 約 -16.9 dB — 同一コーパスで実バイナリ jt9 -5 と一致~わずかに優位             |
| FST4      | 1/1 (FST4-60A)              | 0               | 実バイナリ jt9 -7 と 3サブモード中 2つで一致 (FST4-60 は完全一致)            |
| WSPR      | 9/9                         | 0 (5スロット連鎖でも 0) | 50% 交差 約 -31.5 dB — 実 wsprd とセル単位で一致                    |
| JT9       | 7/7                         | 0               | 50% 交差 約 -26.6 dB — 実 jt9 -9 の既定深度を上回る                  |
| JT65      | 参照録音なし                      | 0               | 自前コーパスでは差ゼロ。ただし実バイナリ jt9 -6 にはまだ差がある (未解決)              |
| Q65       | 実 EME 録音 2件                 | サブモード毎に管理       | 10 サブモードで解析値の 0.2~1.4 dB 以内。CQ-AP ありでは WSJT-X の判定に一致~優位 |
| MSK144    | 3/3 (SNR 値も完全一致)            | 0               | 50% 交差 約 -5.2~-5.8 dB — 実 jt9 と 28セル中 25セルが完全一致         |

### 「再現率だけ」はベンチマークではない

2026-08-11 まで、この表には再現率しか載っていませんでした。当時「WSPR 8/8、パリティ達成」と書かれていたデコーダは、**同時に1スロットあたり8個の存在しないコールサインを出していました** — 誤検出率 50%。再現率だけの表では、それを正常なデコーダと区別できません。いまは precision が列として常設されています。

### 数字は「観測」ではなく「表明」

- 各モードの \*\_wsjtx\_samples テストが、上の数字を **assert しています**。下回れば CI が落ちます。
- MFSK\_REQUIRE\_CORPUS=1 — 録音が見つからないときは「スキップ」ではなく「失敗」。静かに緑になる事故を潰すため。
- 「余分なデコード」は必ずしも誤りではありません。FT8 の混雑バンドでは WSJT-X の 8 に対し 20 出ますが、**超過分 12 はすべて JTDX の正解表に載っています** — 発明ではなく、WSJT-X が取り逃した信号です。

速いことより、正しいこと。取り逃さないことと、でっち上げないことを、同時に測っています。

ホスト — 実録音 1スロットの復調時間

| モード                  | スロット長 | 復調時間   |
|----------------------|-------|--------|
| Q65-60D (10 GHz EME) | 60 s  | 0.05 s |
| FT8 (組み込み構成)         | 15 s  | 0.07 s |
| FST4-60A             | 60 s  | 0.08 s |
| FT4                  | 7.5 s | 0.10 s |
| JT9                  | 60 s  | 0.16 s |
| FT8 (ホスト全力構成)        | 15 s  | 0.18 s |
| WSPR                 | 120 s | 0.76 s |
| MSK144               | 15 s  | 0.88 s |

AMD Ryzen 9 9900X (12C/24T) / --release --features full。WSPR が他より遅いのは意図的で、wsprd の DT ピーク探索を入れて約 1 dB の感度と引き換えにしています。

組み込み — 同じ整数パイプライン

| 実行環境               | 復調時間    |
|--------------------|---------|
| Ryzen ホスト (固定小数点)  | 約 6 ms  |
| ESP32-S3 LX7 実機    | 1.19 s  |
| ESP32 Core2 LX6 実機 | 約 2.8 s |

16局が混み合う WSJT-X 参照録音 qso3\_busy.wav、スロット終了後の実測。ホストとマイコンの差はほぼ純粋な CPU 比 (Ryzen 約5 GHz×16コア 対 Xtensa 240 MHz×2コア) で、アルゴリズム上のオーバーヘッドはありません — 両者は同一の整数パイプラインです。

WSPR をマイコンに載せる:1214秒 → 101.6秒

120秒スロットに対し最初は **10.1倍の超過**。独立した8つの改善を積み上げて **スロットの 0.85倍**に収め、9/9 の正解デコードを全段階で維持しました。効いたのは新アルゴリズムではなく、**参照実装 wsprd が既に持っていた候補フィルタ** (minsync2、単独で 4.1倍)。しかも副産物として誤デコードが 0 になりました。

FT8 は2つの構成を持ち、決して混ぜません。ホスト構成 (DecodeDepth::FULL) は研究用の全力設定。組み込み構成 (EMBEDDED) は ESP32 の時間予算に合わせて意図的に軽くした別のデコーダで、劣化版ではありません。FT8 の QSO 折り返し予算はスロット終了後 **約2秒** — その中で復調・ウォーターフォール描画・次スロットの送信準備まで済ませる必要があります。

Ryzen で 6 ミリ秒、ESP32-S3 で 1.19 秒 — 差は純粋な CPU 性能比です。走っているコードは同じものです。

| 行き先           | ターゲット                                          | FFT / 並列化                         | 状態と備考                                                                                        |
|---------------|------------------------------------------------|-----------------------------------|----------------------------------------------------------------------------------------------|
| デスクトップ / サーバ  | x86_64, aarch64                                | rustfft + rayon                   | 既定構成。crates.io から cargo add mfsk-core ですぐ                                                    |
| ブラウザ          | wasm32-unknown-unknown                         | rustfft の WASM SIMD               | +simd128 を明示するのを忘れずに (既定では付きません)                                                             |
| Android / iOS | NDK arm64-v8a ほか                               | NEON                              | C ABI (mfsk-ffi) 経由。Fortran ランタイム不要                                                          |
| マイコン          | no_std + alloc<br>ESP32-S3 / RP2350 / Cortex-M | 呼び出し側が注入<br>(esp-dsp / CMSIS-DSP) | 実運用中。fft-extern + fixed-point                                                                |
| C / C++ から    | staticlib                                      | —                                 | mfsk-ffi (全モード) と mfsk-ffi-ft8 (FT8 のみ・組み込み向け)。<br>ヘッダは cbindgen 生成、C++ 実ドライバを CI が毎回動かしています |

### 数値型を差し替える

DSP と FEC のパイプライン全体が**スカラ型トレイト**で抽象化されています。ホストは f32、マイコンは u16 スペクトログラム + i16 DFT + Q11i16 LLR + 整数 BP。コードは**同じ一本**です。

### FFT を差し替える

FFT はトレイト越しの外部注入。ホストは rustfft、ESP32 は esp-dsp のアセンブラ、Cortex-M は CMSIS-DSP — ライブラリ側は「誰が計算したか」を知りません。

### 使うモードだけ積む

モードは全部 feature フラグ。FT8 だけ欲しければ FT8 だけがバイナリに入ります。**12通りの feature 組み合わせ**を CI が毎回ビルドして、「full でしかビルドが通らない」事故を防いでいます。

行き先が変わっても書き直しは起きません。変わるのは feature フラグと、外から渡す FFT だけです。

## 約150 KB

FT8 復調に必要な使用可能 RAM  
(FFT を呼び出し側が用意する場合)

## 150–200 KB

ライブラリ分のフラッシュ使用量  
(Core2 実測、アプリのグルー込み)

## 0 バイト

内部 DRAM の常駐スクラッチ  
／要求する extern シンボル

### ライブラリが呼び出し側に求めるもの

- **FFT の実装ひとつ** — fft-extern でファクトリ関数を1つ提供するだけ。ESP32 なら esp-dsp をそのまま使えます。
- **アロケータ** — no\_std + alloc。スロット単位の作業領域は PSRAM で構いません。
- それ以外は何もありません。**スクラッチバッファの配置に悩む必要はもうありません** — 0.8.0 で最後の外部スクラッチ引数を削除しました。

### スロットあたりの作業領域

|               |                    |
|---------------|--------------------|
| スペクトログラム      | 約 360 KB (PSRAM 可) |
| 候補セット         | 約 120 KB           |
| LDPC BP スクラッチ | 約 12 KB            |

### 教訓: 120 KB を返してもらった話

初期実装は FT8 のトーン基底を Q15 の sin/cos 表として持っていました。アセンブラの内積が本来の速度を出すには、この表が**高速な内部 SRAM**に居る必要がある — 30 KB × 2軸 × 2コア = **120 KB の内部 DRAM**。そしてそれは、M5StickS3 の双方向 I2S DMA が必要とする領域そのものでした。QSO モードは起動に失敗しました。

解決は Goertzel 再帰への置き換え。**スクラッチ 0、速度は据え置き、精度はむしろ +0.16~+0.63 dB 改善**。表を消したら全部よくなりました。

### 動いている実機

- **M5Stack CoreS3** (ESP32-S3 LX7) — メインの受信機。USB Audio で無線機直結、4受信機を1イメージに。
- **M5StickS3** (ESP32-S3 LX7) — デモ／音響入力機。LCD UI、BLE CI-V、QSO ステートマシン。
- **M5Stack Core2** (ESP32 LX6) — LX6 側の検証機。USB ペリフェラルを持たないので WAV 再生での確認用。

呼び出し側が用意するのは FFT ひとつとアロケータだけ。スクラッチの置き場所で悩む必要はもうありません。



## プロトコル = ゼロサイズ型

1つのモードは **実行時に1バイトも占めない型**として書かれ、その上に3つのトレイトを実装します。

|                  |                              |
|------------------|------------------------------|
| ModulationParams | トーン数、シンボルレート、Gray 写像、GFSK 整形 |
| FrameLayout      | シンボル数、スロット長、同期ブロック配置         |
| Protocol         | どの FEC と、どのメッセージ符号を使うか       |

共通パイプラインはすべて P: Protocol にジェネリックで、コンパイル時に各モードへ**単相化**されます。**vtable なし、動的ディスパッチなし** — 抽象化のコストはコンパイル時間だけで、実行時にはゼロです。

実測 (2026-08-14) :src/ の約3分の1、58.9k 行中 18.7k 行が P: Protocol にジェネリックなコードです。LDPC の BP/OSD カーネル、メッセージ符号、FFT/リサンプル/GFSK の DSP。

### 共通 (engine、全モードで1本)

粗同期、精密同期、LLR 計算、等化、復調パイプライン本体、GFSK 合成

### モード固有 (型の上の定数として宣言)

トーン数・シンボルレート・Gray 写像・フレーム構成、そして どの FEC (LDPC / 畳み込み Fano / Reed-Solomon / GF(64) 上の QRA) とどのメッセージ幅 (77 / 72 / 50 bit) を使うか

**FST4-60A の追加は、  
ZST 1つへの trait 実装 + レジストリ1行でした。**  
共通コードには一切触れていません。

Fortran のコードベースでは、モード間の「同じであり得たはずの部分」はソースファイル間のコピー＆ペーストとして現れます。ここではそれをトレイトとして書き、**何がモードごとに本当に違うのか**で分割しています。

**モードを1つ足す作業が、共通コードを1行も触らずに終わる — それがこの設計の目的です。**

## 手が足りていないところ

- **ブラウザで動く FT8 — WASM PWA のフロントエンド。**  
ライブラリ側は wasm32 で動きます。UI とオーディオ入力を書く人がいません。
- **Android / iOS アプリ。**  
C ABI は用意済み。JNI の骨組みも置いてあります。
- **ESP32 以外のマイコン。**  
RP2350 + CMSIS-DSP、Cortex-M — FFT を1つ渡せば動くはずですが、まだ誰も試していません。
- **CoreS3 の送信側。**  
受信は動いています。残りは BLE CI-V、ADIF ログ、送信キーイング。
- **JT65 の残差を詰める。**  
自前コーパスでは差ゼロですが、実バイナリにはまだ届いていません。原因未特定 — 面白い問題です。

## まず読むもの

|                            |                           |
|----------------------------|---------------------------|
| README.md                  | 全体像と Quick Start          |
| docs/reference/LIBRARY.md  | ホスト API の全体 (日本語版あり)      |
| docs/reference/EMBEDDED.md | 組み込み・固定小数点・C ABI (日本語版あり) |
| docs/notes/BENCHMARKS.md   | 本日の数字の出どころすべて             |
| CONTRIBUTING.md            | テスト方針と WSJT-X 移植のルール      |

docs/reference/ の文書はすべて日本語版 (.ja.md) を持ちます。



リポジトリ

**github.com/jl1nie/mfsk-core**

crates.io / docs.rs — mfsk-core (最新リリース v0.9.1、0.10.0 準備中)

Issue も PR も歓迎します。

ブースで直接どうぞ — **JL1NIE**

本ライブラリは WSJT-X (K1JT Joe Taylor 氏および共同開発者の皆さん) の Rust 再実装です。すべてのアルゴリズムの出典は WSJT-X にあり、基準実装はこれからも WSJT-X です。

**電波を受けるコードが、Rust で、オープンに、誰の手元にもある状態を目指しています。**